

MARAN PMS — Prompt 58: Night Audit Automático

Objetivo

Automatizar el proceso de cierre nocturno: postear el cargo de alojamiento de cada habitación ocupada, verificar prepagos del día siguiente, y generar un reporte de cierre. Se ejecuta automáticamente a las 00:00 hora Argentina, y también se puede ejecutar manualmente desde Administración.

Nota importante

El bloqueo de operaciones retroactivas NO se implementa en esta versión. Se puede agregar en una etapa posterior cuando el equipo esté cómodo con el sistema.

Archivos a crear/modificar

- `shared/schema.ts` — nueva tabla `night_audit_logs`
 - `server/night-audit.ts` — archivo nuevo con toda la lógica
 - `server/index.ts` — registrar el scheduler
 - `server/routes.ts` — 3 endpoints nuevos
 - `client/src/pages/administration.tsx` — nuevo tab "Night Audit"
-

PARTE 1 — Schema (`shared/schema.ts`)

Agregar al final del archivo:

```
// ===== NIGHT AUDIT =====

export type NightAuditStatus = "success" | "partial" | "failed";

export const nightAuditLogs = pgTable("night_audit_logs", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
```

```

// Fecha auditada (la noche que se procesó)
auditDate: date("audit_date").notNull(),

// Ejecución
executedAt: timestamp("executed_at").notNull().defaultNow(),
executedBy: text("executed_by").notNull().default("sistema"), // "sistema" o no
isManual: boolean("is_manual").notNull().default(false),

// Resultados de posteo de cargos
reservationsProcessed: integer("reservations_processed").notNull().default(0),
reservationsSkipped: integer("reservations_skipped").notNull().default(0),
totalPosted: decimal("total_posted", { precision: 12, scale: 2 }).notNull().def

// Verificación prepagos día siguiente
arrivalsNextDay: integer("arrivals_next_day").notNull().default(0),
arrivalsWithPrepago: integer("arrivals_with_prepago").notNull().default(0),
arrivalsWithoutPrepago: integer("arrivals_without_prepago").notNull().default(0

// Estado general
status: text("status").$type<NightAuditStatus>().notNull().default("success"),
notes: text("notes"),

// Detalle completo en JSON
detail: text("detail"), // JSON stringificado con el detalle por reserva

createdAt: timestamp("created_at").defaultNow(),
});

export const insertNightAuditLogSchema = createInsertSchema(nightAuditLogs).omit(
  id: true,
  createdAt: true,
});

export type InsertNightAuditLog = z.infer<typeof insertNightAuditLogSchema>;
export type NightAuditLog = typeof nightAuditLogs.$inferSelect;

```

Agregar también este campo a `charges` si no existe (verificar en el schema actual):

```

// En charges, agregar:
source: text("source").default("manual"),
// "manual" | "night_audit" | "system"
// Permite identificar qué cargos fueron posteados por el night audit

```

Si `charges` ya tiene `status` y `anulado` en la versión actual del sistema, usarlos. Si no, el

night audit usa los campos existentes sin modificar la tabla `charges` .

PARTE 2 — Lógica del Night Audit (server/night-audit.ts)

Crear archivo nuevo:

```
import { db } from "../db";
import { randomUUID } from "crypto";
import { eq, and, inArray, sql } from "drizzle-orm";
import {
  reservations, charges, payments, guests, rooms,
  nightAuditLogs, systemNotifications,
} from "@shared/schema";
import { log } from "../index";

// Obtener fecha de Argentina como string YYYY-MM-DD
function getArgentinaDate(offsetDays = 0): string {
  const now = new Date();
  now.setDate(now.getDate() + offsetDays);
  return now.toLocaleDateString("en-CA", {
    timeZone: "America/Argentina/Buenos_Aires",
  });
}

// Verificar si ya se ejecutó el night audit para una fecha
export async function nightAuditAlreadyRan(auditDate: string): Promise<boolean> {
  const existing = await db
    .select({ id: nightAuditLogs.id })
    .from(nightAuditLogs)
    .where(eq(nightAuditLogs.auditDate, auditDate))
    .limit(1);
  return existing.length > 0;
}

// Función principal del Night Audit
export async function runNightAudit(options: {
  executedBy?: string;
  isManual?: boolean;
  forceDate?: string; // Para ejecutar manualmente en una fecha específica
}): Promise<{
  success: boolean;
  message: string;
  data?: any;
}> {
```

```

const auditDate = options.forceDate || getArgentinaDate(-1);
// La noche auditada es AYER (la noche que acaba de terminar)
const tomorrow = getArgentinaDate(1);
const executedBy = options.executedBy || "sistema";
const isManual = options.isManual || false;

log(`[NightAudit] Iniciando audit para fecha: ${auditDate}`, "night-audit");

// Verificar si ya se ejecutó (solo para ejecución automática)
if (!isManual) {
  const alreadyRan = await nightAuditAlreadyRan(auditDate);
  if (alreadyRan) {
    log(`[NightAudit] Ya se ejecutó para ${auditDate}, saltando`, "night-audit")
    return { success: true, message: `Night audit ya ejecutado para ${auditDate}` }
  }
}

const detail: any[] = [];
let reservationsProcessed = 0;
let reservationsSkipped = 0;
let totalPosted = 0;
let auditStatus: "success" | "partial" | "failed" = "success";
const errors: string[] = [];

try {
  // =====
  // PASO 1: Postear cargo de alojamiento por cada hab. ocupada
  // =====

  // Obtener reservas checked_in cuya estadía incluye la noche auditada
  // La noche auditada es: checkInDate <= auditDate < checkOutDate
  const activeReservations = await db
    .select()
    .from(reservations)
    .where(
      and(
        eq(reservations.status, "checked_in"),
        sql`${reservations.checkInDate} <= ${auditDate}`,
        sql`${reservations.checkOutDate} > ${auditDate}`
      )
    );

  log(`[NightAudit] Reservas activas encontradas: ${activeReservations.length}`

  for (const reservation of activeReservations) {
    try {
      // Verificar si ya se posteó este cargo para esta noche

```

```

const existingCharge = await db
  .select({ id: charges.id })
  .from(charges)
  .where(
    and(
      eq(charges.reservationId, reservation.id),
      eq(charges.date, auditDate),
      eq(charges.category, "room"),
      sql`${charges.description} LIKE '%Night Audit%'`
    )
  )
  .limit(1);

if (existingCharge.length > 0) {
  detail.push({
    reservationId: reservation.id,
    reservationCode: reservation.reservationCode,
    action: "skipped",
    reason: "Cargo ya posteadó para esta noche",
  });
  reservationsSkipped++;
  continue;
}

// Calcular tarifa de la noche
const ratePerNight = parseFloat(
  reservation.finalRatePerNight || reservation.baseRatePerNight || "0"
);

if (ratePerNight <= 0) {
  detail.push({
    reservationId: reservation.id,
    reservationCode: reservation.reservationCode,
    action: "skipped",
    reason: "Tarifa cero o no configurada",
    rate: ratePerNight,
  });
  reservationsSkipped++;
  continue;
}

// Obtener número de habitación para la descripción
const room = reservation.roomId
  ? await db.select({ roomNumber: rooms.roomNumber })
    .from(rooms)
    .where(eq(rooms.id, reservation.roomId))
    .limit(1)

```

```

    : []];

const roomNumber = room[0]?.roomNumber ?? "?";

// Postear el cargo
await db.insert(charges).values({
  id: randomUUID(),
  reservationId: reservation.id,
  description: `Alojamiento Hab. ${roomNumber} - ${auditDate} (Night Audi
  amount: ratePerNight.toFixed(2),
  date: auditDate,
  category: "room",
  createdBy: "night_audit",
});

totalPosted += ratePerNight;
reservationsProcessed++;

detail.push({
  reservationId: reservation.id,
  reservationCode: reservation.reservationCode,
  roomNumber,
  action: "posted",
  amount: ratePerNight,
  date: auditDate,
});

log(`[NightAudit] ✓ Cargo posteadó: ${reservation.reservationCode} - Hab

} catch (err: any) {
  errors.push(`Error en reserva ${reservation.reservationCode}: ${err.messa
  auditStatus = "partial";
  detail.push({
    reservationId: reservation.id,
    reservationCode: reservation.reservationCode,
    action: "error",
    error: err.message,
  });
}
}

// =====
// PASO 2: Verificar prepagos/garantías del día siguiente
// =====

const arrivalsNextDay = await db
  .select()

```

```

    .from(reservations)
    .where(
      and(
        eq(reservations.checkInDate, tomorrow),
        inArray(reservations.status, ["confirmed", "pending", "tentative"] as a
      )
    );

const arrivalsDetail: any[] = [];
let arrivalsWithPrepago = 0;
let arrivalsWithoutPrepago = 0;

for (const arrival of arrivalsNextDay) {
  // Verificar si tiene algún pago registrado
  const [paymentSum] = await db
    .select({ total: sql<number>`COALESCE(SUM(${payments.amount}::numeric), 0
    .from(payments)
    .where(eq(payments.reservationId, arrival.id));

  const totalPaid = Number(paymentSum?.total ?? 0);
  const hasPrepago = totalPaid > 0;

  if (hasPrepago) {
    arrivalsWithPrepago++;
  } else {
    arrivalsWithoutPrepago++;
  }

  arrivalsDetail.push({
    reservationId: arrival.id,
    reservationCode: arrival.reservationCode,
    checkInDate: arrival.checkInDate,
    totalPaid,
    hasPrepago,
    rate: arrival.finalRatePerNight || arrival.baseRatePerNight,
  });
}

// =====
// PASO 3: Guardar registro del audit
// =====

const [auditLog] = await db.insert(nightAuditLogs).values({
  id: randomUUID(),
  auditDate,
  executedAt: new Date(),
  executedBy,

```

```

    isManual,
    reservationsProcessed,
    reservationsSkipped,
    totalPosted: totalPosted.toFixed(2),
    arrivalsNextDay: arrivalsNextDay.length,
    arrivalsWithPrepago,
    arrivalsWithoutPrepago,
    status: auditStatus,
    notes: errors.length > 0 ? errors.join(" | ") : null,
    detail: JSON.stringify({ charges: detail, arrivals: arrivalsDetail }),
  }).returning();

// =====
// PASO 4: Generar notificación interna
// =====

try {
  await db.insert(systemNotifications).values({
    id: randomUUID(),
    type: "hospitality_alert" as any,
    title: `Night Audit ${auditDate} - ${auditStatus === "success" ? "✓ Compl`
    message: `Se postearon ${reservationsProcessed} cargos por ${totalPosted`
    `${arrivalsNextDay.length} llegadas mañana (${arrivalsWithoutPrepago} s`
    area: "all" as any,
    priority: auditStatus === "success" ? "normal" : "high" as any,
    isRead: false,
    createdAt: new Date(),
  });
} catch {
  // No fallar por error en notificación
}

const result = {
  auditDate,
  executedAt: new Date().toISOString(),
  executedBy,
  isManual,
  reservationsProcessed,
  reservationsSkipped,
  totalPosted: totalPosted.toFixed(2),
  arrivals: {
    total: arrivalsNextDay.length,
    withPrepago: arrivalsWithPrepago,
    withoutPrepago: arrivalsWithoutPrepago,
  },
  status: auditStatus,
  errors,

```



```

        auditLogId: auditLog.id,
    };

    log(`[NightAudit] ✓ Completado: ${reservationsProcessed} cargos, ${totalPost}
    return { success: true, message: "Night audit completado", data: result };

} catch (err: any) {
    log(`[NightAudit] ✗ Error crítico: ${err.message}`, "night-audit");

    // Registrar el fallo
    try {
        await db.insert(nightAuditLogs).values({
            id: randomUUID(),
            auditDate,
            executedAt: new Date(),
            executedBy,
            isManual,
            reservationsProcessed,
            reservationsSkipped,
            totalPosted: "0",
            arrivalsNextDay: 0,
            arrivalsWithPrepago: 0,
            arrivalsWithoutPrepago: 0,
            status: "failed",
            notes: err.message,
            detail: JSON.stringify({ error: err.message }),
        });
    } catch {
        // ignorar error al guardar log
    }

    return { success: false, message: `Error en night audit: ${err.message}` };
}

}

// Scheduler interno - se ejecuta cada minuto y verifica si es hora de correr
export function setupNightAuditScheduler(): void {
    log("[NightAudit] Scheduler iniciado", "night-audit");

    setInterval(async () => {
        try {
            const now = new Date();
            // Hora actual en Argentina
            const argTime = new Date(now.toLocaleString("en-US", {
                timeZone: "America/Argentina/Buenos_Aires"
            }));

```

```

const hour = argTime.getHours();
const minute = argTime.getMinutes();

// Ejecutar a las 00:05 (5 minutos después de medianoche para evitar condic
if (hour === 0 && minute === 5) {
  log("[NightAudit] Hora de ejecución alcanzada, corriendo audit automático
  await runNightAudit({ executedBy: "sistema", isManual: false });
}
} catch (err: any) {
  log(`[NightAudit] Error en scheduler: ${err.message}`, "night-audit");
}
}, 60_000); // Verificar cada 60 segundos
}

```

PARTE 3 — Registrar el scheduler (server/index.ts)

Agregar después de `await registerRoutes(httpServer, app) :`

```

// Night Audit scheduler
import { setupNightAuditScheduler } from "../night-audit";

// Dentro del IIFE async, después de registerRoutes:
setupNightAuditScheduler();
log("Night Audit scheduler iniciado");

```

PARTE 4 — Endpoints (server/routes.ts)

Agregar dentro de `registerRoutes :`

```

// ===== NIGHT AUDIT =====

import { runNightAudit, nightAuditAlreadyRan } from "../night-audit";

// Ejecutar night audit manualmente
app.post("/api/night-audit/run", requireAuth, async (req, res) => {
  try {
    const { forceDate, executedBy } = req.body;
    const userName = (req.user as any)?.fullName || (req.user as any)?.username |

    // Verificar si ya se ejecutó para esa fecha
    const targetDate = forceDate || new Date(

```

```

        new Date().toLocaleString("en-US", { timeZone: "America/Argentina/Buenos_Ai
    }).toLocaleDateString("en-CA", { timeZone: "America/Argentina/Buenos_Aires" })

    // Si ya se ejecutó y no se fuerza, advertir
    const alreadyRan = await nightAuditAlreadyRan(forceDate ||
        new Date().toLocaleDateString("en-CA", { timeZone: "America/Argentina/Bueno
    });

    if (alreadyRan && !req.body.force) {
        return res.status(409).json({
            error: "El night audit ya se ejecutó para esta fecha",
            alreadyRan: true,
            message: "Agregá force: true para ejecutar de nuevo"
        });
    }

    const result = await runNightAudit({
        executedBy: userName,
        isManual: true,
        forceDate,
    });

    if (result.success) {
        res.json(result.data);
    } else {
        res.status(500).json({ error: result.message });
    }
} catch (error: any) {
    res.status(500).json({ error: "Error ejecutando night audit: " + error.messag
});

// Historial de night audits
app.get("/api/night-audit/history", requireAuth, async (req, res) => {
    try {
        const limit = parseInt(req.query.limit as string || "30");

        const history = await db
            .select()
            .from(nightAuditLogs)
            .orderBy(desc(nightAuditLogs.executedAt))
            .limit(limit);

        res.json(history);
    } catch (error) {
        res.status(500).json({ error: "Error fetching night audit history" });
    }
}

```

```

});

// Estado del último night audit
app.get("/api/night-audit/status", requireAuth, async (req, res) => {
  try {
    const today = new Date().toLocaleDateString("en-CA", {
      timeZone: "America/Argentina/Buenos_Aires"
    });
    const yesterday = new Date(Date.now() - 86400000).toLocaleDateString("en-CA", {
      timeZone: "America/Argentina/Buenos_Aires"
    });

    const [lastAudit] = await db
      .select()
      .from(nightAuditLogs)
      .orderBy(desc(nightAuditLogs.executedAt))
      .limit(1);

    const todayRan = await nightAuditAlreadyRan(today);
    const yesterdayRan = await nightAuditAlreadyRan(yesterday);

    res.json({
      today,
      yesterday,
      lastAudit: lastAudit || null,
      todayRan,
      yesterdayRan,
      nextScheduled: "00:05 hora Argentina",
    });
  } catch (error) {
    res.status(500).json({ error: "Error fetching night audit status" });
  }
});

```

Agregar imports en routes.ts:

```

import { nightAuditLogs } from "@shared/schema";
// desc ya debería estar importado de drizzle-orm

```

PARTE 5 — Frontend (client/src/pages/administration.tsx)

5.1 Nuevo tab en TabsList

```

<TabsTrigger value="night-audit" data-testid="tab-night-audit">
  <Moon className="h-4 w-4 mr-1" />
  Night Audit
</TabsTrigger>

```

5.2 TabsContent

```

<TabsContent value="night-audit" className="space-y-4">
  <NightAuditTab />
</TabsContent>

```

5.3 Componente NightAuditTab

```

function NightAuditTab() {
  const { toast } = useToast();
  const [isRunning, setIsRunning] = useState(false);
  const [forceDate, setForceDate] = useState("");
  const [lastResult, setLastResult] = useState<any>(null);
  const [showConfirm, setShowConfirm] = useState(false);

  const { data: status, refetch: refetchStatus } = useQuery<any>({
    queryKey: ["/api/night-audit/status"],
    queryFn: async () => {
      const res = await fetch("/api/night-audit/status", { credentials: "include" });
      if (!res.ok) throw new Error("Error");
      return res.json();
    },
    refetchInterval: 60_000,
  });

  const { data: history = [], refetch: refetchHistory } = useQuery<any[]>({
    queryKey: ["/api/night-audit/history"],
    queryFn: async () => {
      const res = await fetch("/api/night-audit/history?limit=20", { credentials: "include" });
      if (!res.ok) throw new Error("Error");
      return res.json();
    },
  });

  const runAudit = async (force = false) => {
    setIsRunning(true);
    setShowConfirm(false);
    try {
      const body: any = { force };

```

```

    if (forceDate) body.forceDate = forceDate;

    const res = await fetch("/api/night-audit/run", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "include",
      body: JSON.stringify(body),
    });

    if (res.status === 409) {
      // Ya se ejecutó - pedir confirmación para forzar
      setShowConfirm(true);
      setIsRunning(false);
      return;
    }

    if (!res.ok) {
      const err = await res.json();
      throw new Error(err.error || "Error desconocido");
    }

    const data = await res.json();
    setLastResult(data);
    refetchStatus();
    refetchHistory();
    queryClient.invalidateQueries({ queryKey: ["/api/night-audit"] });
    toast({
      title: "Night Audit completado",
      description: `${data.reservationsProcessed} cargos posteados por ${Numbe
    });
  } catch (err: any) {
    toast({
      title: "Error en Night Audit",
      description: err.message,
      variant: "destructive",
    });
  } finally {
    setIsRunning(false);
  }
};

const STATUS_COLOR: Record<string, string> = {
  success: "text-green-600",
  partial: "text-yellow-600",
  failed: "text-red-600",
};

```

```

const STATUS_LABEL: Record<string, string> = {
  success: "Exitoso",
  partial: "Parcial",
  failed: "Fallido",
};

return (
  <div className="space-y-6">

    {/* Estado actual */}

    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">

      <Card className={`border-l-4 ${status?.yesterdayRan ? "border-l-green-400"
        <CardContent className="p-4">
          <div className="flex items-center gap-2 mb-1">
            {status?.yesterdayRan
              ? <CheckCircle className="h-4 w-4 text-green-600" />
              : <AlertTriangle className="h-4 w-4 text-red-600" />
            }
            <span className="text-sm font-medium">Anoche</span>
          </div>
          <p className={`text-sm ${status?.yesterdayRan ? "text-green-600" : "t
            {status?.yesterdayRan ? "Ejecutado correctamente" : "⚠ No se ejecut
          </p>
        </CardContent>
      </Card>

      <Card>
        <CardContent className="p-4">
          <div className="flex items-center gap-2 mb-1">
            <Clock className="h-4 w-4 text-muted-foreground" />
            <span className="text-sm font-medium">Próxima ejecución</span>
          </div>
          <p className="text-sm text-muted-foreground">
            {status?.nextScheduled ?? "00:05 hora Argentina"}
          </p>
        </CardContent>
      </Card>

      <Card>
        <CardContent className="p-4">
          <div className="flex items-center gap-2 mb-1">
            <Moon className="h-4 w-4 text-muted-foreground" />
            <span className="text-sm font-medium">Último audit</span>
          </div>
          {status?.lastAudit ? (
            <div>

```

```

        <p className={`text-sm font-medium ${STATUS_COLOR[status.lastAudit.status]} - {status.lastAudit.aud
        {STATUS_LABEL[status.lastAudit.status]} - {status.lastAudit.aud
    </p>
    <p className="text-xs text-muted-foreground">
        {new Date(status.lastAudit.executedAt).toLocaleString("es-AR")}
    </p>
    </div>
    ) : (
        <p className="text-sm text-muted-foreground">Sin registros</p>
    )}
    </CardContent>
    </Card>

</div>

{/* Ejecución manual */}
<Card>
    <CardHeader className="pb-3">
        <CardTitle className="text-sm">Ejecución manual</CardTitle>
    </CardHeader>
    <CardContent className="space-y-4">
        <p className="text-sm text-muted-foreground">
            El night audit se ejecuta automáticamente a las 00:05. Podés ejecutar
        </p>

        <div className="flex items-end gap-3">
            <div>
                <Label className="text-xs">Fecha a auditar (dejar vacío = anoche)</
                <Input
                    type="date"
                    value={forceDate}
                    onChange={e => setForceDate(e.target.value)}
                    className="w-44"
                    data-testid="input-audit-date"
                />
            </div>
            <Button
                onClick={() => runAudit(false)}
                disabled={isRunning}
                data-testid="btn-run-night-audit"
            >
                {isRunning ? (
                    <>
                        <RefreshCw className="h-4 w-4 mr-2 animate-spin" />
                        Ejecutando...
                    </>
                ) : (

```



```

        <>
        <Moon className="h-4 w-4 mr-2" />
        Ejecutar Night Audit
      </>
    )}
  </Button>
</div>

{/* Confirmación para re-ejecutar */}
{showConfirm && (
  <div className="p-3 bg-yellow-50 dark:bg-yellow-900/20 border border-
    <p className="text-sm font-medium text-yellow-800 dark:text-yellow-
      ▲ Ya se ejecutó el night audit para esta fecha
    </p>
    <p className="text-xs text-yellow-700 dark:text-yellow-500">
      ¿Quieres ejecutarlo de nuevo? Esto puede generar cargos duplicados
    </p>
    <div className="flex gap-2">
      <Button size="sm" variant="destructive" onClick={() => runAudit(t
        Ejecutar de todas formas
      </Button>
      <Button size="sm" variant="outline" onClick={() => setShowConfirm
        Cancelar
      </Button>
    </div>
  </div>
)}
</CardContent>
</Card>

{/* Resultado del último audit ejecutado en esta sesión */}
{lastResult && (
  <Card className="border-green-200 dark:border-green-800">
    <CardHeader className="pb-3">
      <CardTitle className="text-sm flex items-center gap-2 text-green-700
        <CheckCircle className="h-4 w-4" />
        Resultado — {lastResult.auditDate}
      </CardTitle>
    </CardHeader>
    <CardContent>
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mb-4">
        {[
          { label: "Cargos posteados", value: lastResult.reservationsProces
          { label: "Total posteados", value: `${Number(lastResult.totalPost
          { label: "Saltados", value: lastResult.reservationsSkipped },
          { label: "Llegadas mañana", value: lastResult.arrivals?.total ??
        ].map(({ label, value }) => (

```

```

        <div key={label} className="text-center">
            <p className="text-2xl font-bold">{value}</p>
            <p className="text-xs text-muted-foreground">{label}</p>
        </div>
    )))
</div>

{ (lastResult.arrivals?.withoutPrepago ?? 0) > 0 && (
    <div className="p-2 bg-amber-50 dark:bg-amber-900/20 rounded-md">
        <p className="text-sm text-amber-700 dark:text-amber-400 flex ite
            <AlertTriangle className="h-4 w-4" />
            {lastResult.arrivals.withoutPrepago} llegada(s) para mañana sin
        </p>
    </div>
)}

{lastResult.errors?.length > 0 && (
    <div className="mt-2 p-2 bg-red-50 dark:bg-red-900/20 rounded-md">
        <p className="text-xs text-red-600 font-medium">Errores:</p>
        {lastResult.errors.map((e: string, i: number) => (
            <p key={i} className="text-xs text-red-600">{e}</p>
        ))}
    </div>
)}
</CardContent>
</Card>
)}

{/* Historial */}
<Card>
    <CardHeader className="pb-3">
        <CardTitle className="text-sm">Historial de ejecuciones</CardTitle>
    </CardHeader>
    <CardContent className="p-0">
        {history.length === 0 ? (
            <p className="text-sm text-muted-foreground p-4 text-center">Sin regi
        ) : (
            <Table>
                <TableHeader>
                    <TableRow>
                        <TableHead>Fecha auditada</TableHead>
                        <TableHead>Ejecutado</TableHead>
                        <TableHead>Por</TableHead>
                        <TableHead className="text-center">Cargos</TableHead>
                        <TableHead className="text-right">Total</TableHead>
                        <TableHead className="text-center">Llegadas mañana</TableHead>
                        <TableHead>Estado</TableHead>

```

```

        </TableRow>
    </TableHeader>
    <TableBody>
        {history.map((audit: any) => (
            <TableRow key={audit.id}>
                <TableCell className="font-mono text-sm">{audit.auditDate}</T
                <TableCell className="text-xs text-muted-foreground">
                    {new Date(audit.executedAt).toLocaleString("es-AR")}
                    {audit.isManual && (
                        <Badge variant="outline" className="ml-1 text-[10px]">man
                    )}
                </TableCell>
                <TableCell className="text-sm">{audit.executedBy}</TableCell>
                <TableCell className="text-center text-sm">{audit.reservation
                <TableCell className="text-right text-sm font-medium">
                    ${Number(audit.totalPosted).toLocaleString("es-AR")}
                </TableCell>
                <TableCell className="text-center text-sm">
                    {audit.arrivalsNextDay}
                    {audit.arrivalsWithoutPrepago > 0 && (
                        <Badge className="ml-1 text-[10px] bg-amber-100 text-ambe
                            {audit.arrivalsWithoutPrepago} sin prepago
                        </Badge>
                    )}
                </TableCell>
                <TableCell>
                    <Badge className={`text-xs ${
                        audit.status === "success" ? "bg-green-100 text-green-700
                        audit.status === "partial" ? "bg-yellow-100 text-yellow-7
                        "bg-red-100 text-red-700"
                    }`} >
                        {STATUS_LABEL[audit.status] ?? audit.status}
                    </Badge>
                </TableCell>
            </TableRow>
        )})}
    </TableBody>
</Table>
    )}
</CardContent>
</Card>

</div>
);
}

```

5.4 Imports adicionales necesarios en `administration.tsx`

```
import { Moon, Clock, CheckCircle, AlertTriangle, RefreshCw } from "lucide-react"
// Badge, Table, TableBody, TableCell, TableHead, TableHeader, TableRow — verific
```

NOTAS TÉCNICAS

Sobre el scheduler sin `node-cron`: El sistema usa `setInterval` cada 60 segundos en lugar de una librería de cron. Es más simple y no requiere dependencias adicionales. La desventaja es que si el servidor se reinicia entre las 00:00 y las 00:06, no se ejecuta hasta el día siguiente — en ese caso se puede ejecutar manualmente.

Protección contra duplicados: Antes de postear cada cargo, el sistema verifica si ya existe un cargo de tipo `room` con descripción que contenga "Night Audit" para esa fecha y reserva. Esto protege contra doble ejecución.

El campo `createdBy: "night_audit"`: Permite identificar en el folio de la reserva qué cargos fueron automáticos vs manuales.

Idempotencia: Si se ejecuta dos veces para la misma fecha, la segunda ejecución va a saltar todos los cargos ya posteados y reportar 0 cargos procesados. La confirmación en la UI advierte sobre esto.

Migración DB:

```
drizzle-kit push
```

Crea la tabla `night_audit_logs`.

Consideración futura — bloqueo retroactivo: Cuando se decida implementar, agregar un campo `lockedDate` en `system_settings` que indique hasta qué fecha están bloqueadas las modificaciones. El night audit actualizaría ese campo automáticamente.